

Homework 2: Antimicrobial Property prediction with tree ensembles and graph neural networks

Submission guidelines:

- The homework is due on **July 5rd**.
- Homework is to be submitted alone or by pairs.
- Answers are to be submitted in English.
- A Google Colab notebook to get started is provided here:
https://colab.research.google.com/drive/175K_tqNhHlc8ZiFXUEqIqbl2vEbPX3A_?usp=sharing

Make sure to select GPU runtime for faster training of GNN.

Please submit your solution as a Google Colab notebook. Use markdown cells to write your answers to the questions and export the notebook as a pdf (with Google Colab: File -> Print -> Save as PDF).

Guidelines on using LLMs (ChatGPT-like):

The purpose is (obviously) to learn about the tools studied in class rather than get a good grade 😊. As, such, LLM usage is only partially allowed:

- It is not permitted to directly prompt the model with the homework questions.
 - Agents are not permitted except when specified (DeepSearch, Agent mode of Copilot, etc.).
 - It is permitted to use LLMs to edit your answers for the sake of improving their readability.
 - It is permitted to use LLMs for troubleshooting errors and code completion.
 - It is permitted to use LLMs for generating working toy examples of the various functions called.
- However, you will learn more by spending time on the package documentations (scikit-learn, etc.).

Background:

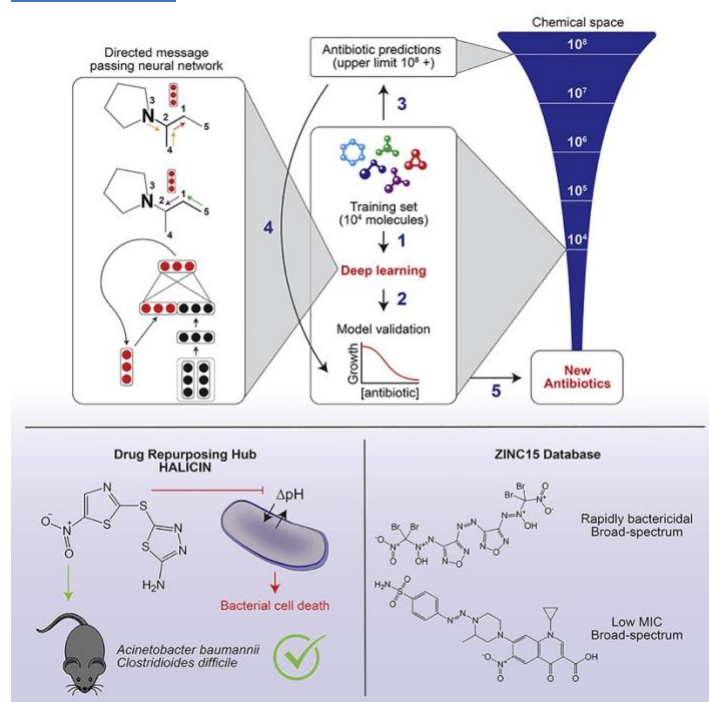


Figure from: [A Deep Learning Approach to Antimicrobial Discovery](#), Stokes et al. Cell 2020.

Antimicrobial resistance: A growing public health threat is the development of bacterial resistance to standard bactericidal treatments (e.g., penicillin, etc.). For example, certain strains of

Staphylococcus Aureus can produce beta-lactamase, an enzyme that can cleave penicillin, and preventing it from killing the bacteria. Every time a new antibacterial molecule is gets into the market, resistance starts to emerge after a couple of years. High-throughput discovery of safe molecules with bactericide activity is thus a growing need.

High-throughput screening: Experimentally testing the antimicrobial activity of a molecule against a certain bacterial strain is a relatively simple assay that can be run in parallel for a few thousand molecules using bacterial growth assays in Petri dishes, scanners and automated image recognition. However, experimental methods are insufficient for testing the activity of the hundreds of millions of known molecules (Zinc database), or for repurposing existing drugs that are known to be safe (Drug Repurposing Hub database).

Machine learning for antimicrobial discovery: A promising synergistic experimental/computational approach was described in the article [A Deep Learning Approach to Antimicrobial Discovery, Stokes et al. Cell 2020](#). It consists of the following steps:

1. Select an initial set of a few thousand candidate molecules for experimental screening.
2. Experimentally screen them for antimicrobial activity against a certain bacterial strand (E-Coli).
3. Build a Machine Learning model that takes as input the molecular graph of the molecule and predicts whether it has high antimicrobial activity or not.
4. Use the Machine Learning model to predict the activity of a very large set of small molecules and curate a set of hits with high predicted activity and low similarity to known antimicrobials.
5. Experimentally test the top-hits against the strand of interest and other strands.

Goal:

Our objective is to replicate Step 3 using the same dataset as in the article. We will train:

- A feature-based model, based on handcrafted chemical descriptors and tree ensembles. We will explain the model using post-hoc explanations tools studied in class.
- A graph neural network architecture adapted from the original article. Each node (resp. edge) will correspond to an atom (resp. covalent bond) of the molecule.

Dataset: A notebook is provided to load the dataset and conduct basic analysis.

Part I: Exploratory Data Analysis

For this section only, analyze the data using the entire dataset.

1. Display, as a scatter plot, the distribution of the number of atoms and covalent bonds per molecule. Color by the predicted class. Is there a relationship between molecule size and activity?
2. For each of the following five features:
 - a. Octanol-water partition coefficient
 - b. Molecular weight
 - c. Number of hydrogen bond donors
 - d. Number of hydrogen bond acceptors
 - e. Number of aromatic rings

Display the histogram of the feature value, for each of the two classes. Which feature(s) have the strongest discriminative power?

Part II: Splitting the data

Our goal is to build a model that can extrapolate well to unseen molecules. To this end, we want to make sure that our molecules from the train, validation and test are “dissimilar enough”. Here, we use the [Tanimoto similarity](#) to calculate similarity between molecules (from 0 = dissimilar to 1 = maximally similar).

3. Go over the notebook and justify the choice of threshold used for Tanimoto similarity.
4. Explain how the algorithm prevents two molecules with high Tanimoto similarity from ending up in the same split.

Part III: Training a tree ensemble model on the Dataset.

Throughout the rest of the homework, the metric for model selection and evaluation is chosen to be the Area under the Precision Recall Curve (implemented as the “average_precision_score” in sklearn).

5. Justify this choice of metric given the problem considered.
6. Using sklearn’s HistGradBoosting (or xgboost/LightGBM), build a Gradient Boosted Tree classifier. Select the optimal hyperparameters (number of iterations; learning rate; regularization strength; number of leaves) by cross-validation using the provided group split. Calculate the precision-recall curve over the test set, display it and report the AUCPR. Comment on the quality of the predictions.

Part IV: Post-hoc explanations of the tree ensemble model

7. Using sklearn, calculate the feature importance using the Permutation Feature Importance metric over the train and test set. Which features are the most informative?
8. Using sklearn, display the Partial Dependence Plots (PDP) for the 10 most important features, ordered by importance. Comment on the findings.
9. Predict the activity of [Halicin](#) (SMILE representation: Nc1nnc(Sc2ncc([N+](=O)[O-])s2)s1), [Amoxicillin](#) (SMILE representation: CC1(C)SC2C(NC(=O)C(N)c3ccc(O)cc3)C(=O)N2C1C(=O)O) and calculate the Shapley values associated to these prediction using shap. Are they predicted to be active and why?
10. Calculate the Shapley values for all active molecules in the test set and produce a low-dimensional embedding using t-SNE. Do you find that there are clusters of molecules with similar Shapley values? Are they known to have a common mechanism of action? (you are encouraged to use ChatGPT/GeminiPro/Grok’s DeepResearch for this).

Part V: Training a Graph Neural Network

We now turn to building and training a Graph Neural Network using the [PyTorch Geometric](#) package. I recommend you to start by going over the following [example notebook](#). A code for training and evaluating a basic Graph Convolution Network (GCN) is already provided.

11. **Setting the number of epochs and early stopping.**
 - a. Modify the code to record the Area Under the Precision Recall Curve (AUCPR) over the train and validation set after each epoch and plot the learning curve (metric vs epoch) for

the train and validation set. Adjust the number of epochs until overfitting is observed.

- b. Retrain the same GCN model, but add an early stopping criterion using the AUCPR calculated over the validation set. Report the performance on the test set. How does it compare to the feature-based model?
12. How many parameters does the GCN have, as function of the number of layers and of the dimension of the node features?
13. **Varying the network width:** Train GCNs with varying node feature dimensions ([16,32,64,128]) and plot the learning curves of the model (loss vs number of epochs) on the training and validation sets. Are there signs of overfitting? Why?
14. **Varying the network depth and the oversmoothing effect:** Train GCNs with varying number of layers ([1, 3,5,10]), and (approximately) fixed number of parameters, and plot the learning curves of the model. Are there signs of overfitting? Underfitting? Why?
15. **Virtual screening:** Pick the best GNN, and use it to predict antimicrobial activity on the evaluation set. How do the predictions correlate with the ones of the D-MPNN model? Does it predict halicin to have an activity?

Bonus: So far, we did not use at all the edge features (i.e., whether a chemical bond is a simple, double, aromatic bond, etc.). Build a Message-Passing Neural Network that integrate these features and compare the performance of the model with the one of the GCN.